

1. Введение и цели работы

В рамках индивидуального задания (itask) была проведена модификация механизмов взаимодействия между пользовательским пространством (Ring 3) и ядром (Ring 0) в ОС JOS. Традиционный механизм системных вызовов в JOS основан на программном прерывании `int 0x30`, которое является «тяжелым» из-за автоматического сохранения множества регистров аппаратным обеспечением, обращения к таблице дескрипторов прерываний (IDT) и сложных проверок сегментации.

Целью работы стала реализация альтернативного механизма на базе инструкций `syscall/sysret`, специфичных для архитектуры x86-64. Эти инструкции обеспечивают переход с минимальными задержками, так как они полагаются на плоскую модель памяти, игнорируют большинство проверок сегментации и используют модельно-специфичные регистры (MSR) для хранения точек входа.

2. Реализация на уровне ядра

Файл `inc/memlayout.h`

В этот файл были добавлены определения новых селекторов сегментов:

- `GD_SYSRET_BASE`: Базовый индекс в GDT, используемый для вычисления селекторов при возврате в пользовательский режим.
- `GD_UD_SYSRET` и `GD_UT_SYSRET`: Сегменты данных и кода пользователя для работы инструкции `sysret`.

Файл `kern/trap.c`

В таблицу дескрипторов `gdt` были добавлены соответствующие записи для новых сегментов. В функцию `trap_init_percpu` был добавлен вызов `syscall_fast_init()`.

Файл `kern/syscall_fast.h`

Здесь определена структура `struct SyscallFastFrame`, которая описывает формат кадра данных на стеке, который создается низкоуровневым ассемблерным обработчиком. В неё входят регистры общего назначения, а также сохраненные значения `user_rip`, `user_rflags` и `user_rsp`.

Файл `kern/syscall_fast.c`

Файл содержит высокоуровневую логику настройки и обработки.

- `is_user_canonical(uint64_t va)`: проверка адреса на «каноничность».
- `syscall_fast_init()`: программирование аппаратных механизмов быстрого вызова.
- `syscall_fast_handler(struct SyscallFastFrame *f)`: проверяет каноничность адресов `%rip` и `%rsp`, синхронизирует текущее состояние регистров со структурой `curenv->env_tf`, вызывает общую ядерную функцию `syscall()`, передавая аргументы из кадра, сохраняет результат в поле `rax` кадра для возврата пользователю.

Файл `kern/syscallentry.s` реализует ассемблерный переход между аппаратным вызовом и C-кодом.

3. Интерфейсы

Файл `lib/syscall.c`

Была реализована возможность переключения механизмов.

- `jcs_syscall_mechanism`: Глобальная переменная, определяющая текущий активный метод (`INT` или `SYSCALL`).
- `raw_syscall_sysret(...)`: выполняет инструкцию `syscall`.
- `jcs_syscall(...)`: функция-селектор, которая направляет вызов либо в старую функцию `syscall` (через прерывание), либо в `raw_syscall_sysret`. Все стандартные обертки были изменены для использования этого селектора.

4. Тестирование и измерение накладных расходов

Файл `user/itaskcorrect.c`

Проверяет корректность работы системных вызовов через оба механизма.

Файл `user/itaskoverhead.c`

Измерение накладных расходов.

- `rdtsc_serial()`: Использует инструкцию `rdtsc` с барьером `lfence` для точного подсчета циклов процессора.
- `bench_nop(...)`: Измеряет время выполнения пустого системного вызова `SYS_nop`.